Neural Network for Payment Identity Resolution in Fintech

Overview

This project develops a **Neural Network–based Payment Identity Resolution System** capable of assigning payments made with a **default identification number** (e.g., 9999999999) to the most probable correct payer identification number (TIN).

The solution is designed for fintech, tax, and payment-processing environments where incomplete or placeholder identifiers are frequently used during transactions.

The system uses:

- Data cleaning
- Fuzzy matching
- Feature engineering
- Deep learning (Neural Network)
- Confidence scoring

to intelligently reconcile unidentified payments with registered taxpayers.

Business Problem

In many fintech and payment systems, users make payments using:

- Default TINs
- Placeholder IDs
- Anonymous identifiers
- Incomplete customer records

This creates major challenges:

- Incorrect payment attribution
- Revenue reconciliation issues
- Audit and compliance risks
- Manual reconciliation workload
- Customer service delays

Traditional rule-based matching systems are often ineffective because:

- Names may be misspelled
- Phone numbers may vary

- Addresses may be incomplete
- Data formats are inconsistent

This project solves the problem using machine learning and neural networks.

Objectives

The project aims to:

- Identify payments made with default identifiers
- Match them to probable taxpayers
- Generate confidence scores for predictions
- Reduce manual reconciliation effort
- Improve payment attribution accuracy
- Support audit and compliance processes

Features

Data Cleaning

- Name normalization
- Phone number standardization
- Address cleaning
- Removal of invalid characters

Fuzzy Matching

Uses similarity scoring for:

- Names
- Phone numbers
- Addresses

Neural Network Prediction

Uses TensorFlow/Keras deep learning model to:

- Learn matching patterns
- Predict correct TIN
- Produce confidence scores

Confidence-Based Review

Predictions are categorized as:

- High confidence
- Medium confidence
- Manual review required

Export Results

Outputs:

- Updated payment file
- Review-only file
- Match confidence scores

Technologies Used

Technology	Purpose
Python	Core programming
Pandas	Data processing
NumPy	Numerical computation
TensorFlow / Keras	Neural network
Scikit-learn	Preprocessing & evaluation
RapidFuzz	Fuzzy matching
OpenPyXL	Excel file handling
Jupyter Notebook	Development environment

Project Structure

```
project/
│
├── Neural_Network_Payment_Matching.ipynb
├── README.md
│
├── data/
│   ├── March_Collections.xlsx
│   ├── ITaxpayer1.xlsx
│   ├── ITaxpayer2.xlsx
│   ├── ITaxpayer3.xlsx
│   ├── ITaxpayer4.xlsx
│   ├── ITaxpayer5.xlsx
│   ├── ITaxpayer6.xlsx
│   ├── ITaxpayer7.xlsx
│   ├── ITaxpayer8.xlsx
│   ├── ITaxpayer9.xlsx
│   └── CTaxpayer.xlsx
│
├── output/
│   ├── March_collection_with_probable_KRIN_TIN.xlsx
│   └── Manual_review_required.xlsx
│
└── requirements.txt
```

Installation

Clone Repository

```
git clone https://github.com/yourusername/neural-network-payment-matching.git
cd neural-network-payment-matching
```

Install Dependencies

```
pip install pandas numpy scikit-learn tensorflow openpyxl rapidfuzz
```

Or:

```
pip install -r requirements.txt
```

Dataset Requirements

The datasets should contain columns similar to:

Collections Dataset

Column	Description
TIN	Payer identification number
PAYER NAME	Name of payer
PHONE	Phone number
ADDRESS	Address

Taxpayer Dataset

Column	Description
TIN	Registered taxpayer identifier
TAXPAYER NAME	Taxpayer full name
PHONE	Registered phone number
ADDRESS	Registered address

How It Works

Step 1 — Load Data

The system loads:

- Collections data
- Individual taxpayer records
- Corporate taxpayer records

Step 2 — Clean Data

The model standardizes:

- Names
- Phone numbers

- Addresses

Step 3 — Detect Default TIN Payments

Transactions using:

`9999999999`

are isolated for processing.

Step 4 — Generate Candidate Matches

The system computes:

- Name similarity
- Phone similarity
- Address similarity

using fuzzy matching algorithms.

Step 5 — Train Neural Network

The neural network learns:

- Matching patterns
- Relationship weights
- Similarity behavior

Step 6 — Predict Probable TIN

The model predicts:

- Most probable taxpayer
- Confidence score

Step 7 — Export Results

The output file contains:

- Predicted TIN
- Confidence score
- Match status

Neural Network Architecture

```
Input Layer
    ↓
Dense Layer (32 neurons, ReLU)
    ↓
Dropout Layer (20%)
    ↓
Dense Layer (16 neurons, ReLU)
    ↓
Dropout Layer (20%)
    ↓
Dense Layer (8 neurons, ReLU)
    ↓
Output Layer (Sigmoid)
```

Example Output

Payment Name	Default TIN	Predicted TIN	Confidence
John Doe	9999999999	1002938475	0.96
Jane Smith	9999999999	1008374652	0.82

Evaluation Metrics

The model uses:

- Accuracy
- Precision
- Recall
- F1-Score

to evaluate prediction quality.

Challenges Encountered

During implementation:

- File naming inconsistencies
- Corrupted Excel files
- Missing columns
- Execution-order issues in Jupyter Notebook

were resolved through:

- File validation
- Dynamic column inspection
- Error handling
- Data preprocessing

Future Improvements

Possible future enhancements:

- Transformer-based matching
- Real-time API integration
- Fraud detection integration
- Incremental learning
- Cloud deployment
- Explainable AI (XAI)

Compliance & Security Considerations

This solution supports:

- Audit traceability
- Data integrity
- Financial reconciliation
- Operational efficiency

Potential regulatory alignment:

- PCI DSS
- SOX
- FFIEC
- GDPR/Data privacy practices

Sample Use Cases

- Fintech reconciliation
- Tax payment systems
- Banking payment recovery
- Government revenue systems
- ERP payment attribution
- Customer identity resolution

Author

Precious Akpokighe

Business Analyst | Product Owner/Manager| AI & Data Analytics | Fintech Solutions

Acknowledgements

Libraries and tools used:

- TensorFlow
- Scikit-learn
- Pandas
- RapidFuzz
- OpenPyXL
- Jupyter Notebook